Autonomous and Mobile Robotics

Wheeled Mobile Robots: Path & Trajectory Planning

Lecture notes

Contents

1 Planning admissible motions *(slides 3)* **1**

2 Space-time separation *(slide 4)* **2**

3 Admissible paths and geometric model *(slides 5–8)* **2**

3.1 Geometric path admissibility *(slides 5–6)* 2

3.2 The unicycle example *(slides 7–8)* 4

4 Flat outputs *(slides 9–13)* **5**

4.1 Differential flatness *(slides 9–10)* 5

4.2 Flatness and the chained form *(slides 11–12)* 7

4.3 An algorithm for planning with flat outputs *(slide 13)* 8

5 Unicycle path planning via Cartesian polynomials *(slides 14–15)* **8**

6 Unicycle path planning via the chained form *(slides 16–17)* **10**

7 Path planning via parametrized inputs *(slides 18–21)* **11**

7.1 Chained form with parametrized inputs *(slides 18–20)* 11

7.2 Other parametrizations and trajectory planning *(slide 21)* 12

8 Parking a unicycle: numerical results *(slides 22–27)* **12**

8.1 Planner based on Cartesian polynomials *(slides 22–24)* 12

8.2 Planner based on parametrized inputs *(slides 25–27)* 13

9 Choice of timing law *(outline item)* **14**

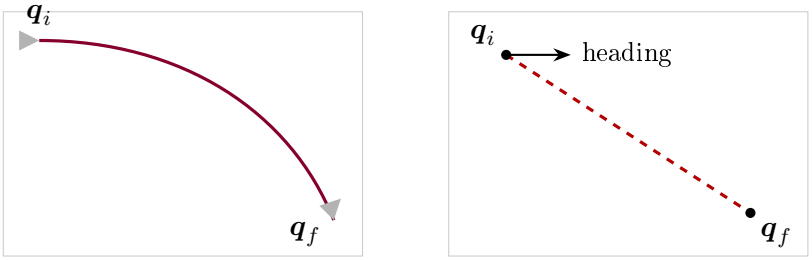
10 Summary **15**

Roadmap. The lecture is about computing motions for a wheeled mobile robot (WMR) that *respect its nonholonomic constraints*, in free space (no obstacles). The plan is: separate space from time; characterize which *paths* are admissible (geometric model); exploit *differential flatness* to generate admissible paths by pure interpolation; apply this to the unicycle via Cartesian polynomials, via the chained form, and via parametrized inputs; look at numerical parking maneuvers; and finally see how a timing law turns a path into a trajectory.

1 Planning admissible motions *(slides 3)*

Definition 1.1 (Motion planning without obstacles)**.** Given an initial configuration $\boldsymbol{q}_i$ and a final configuration $\boldsymbol{q}_f$, find a motion $\boldsymbol{q}(t)$ that transfers the WMR from $\boldsymbol{q}_i$ to $\boldsymbol{q}_f$ and is *admissible*, i.e., satisfies the nonholonomic constraints $A(\boldsymbol{q})\dot{\boldsymbol{q}} = 0$ at every time instant.

Meeting the boundary conditions is not enough: a curve that joins $\boldsymbol{q}_i$ to $\boldsymbol{q}_f$ but demands, say, a sideways slip of the wheels is useless. Admissibility is what makes this problem different from plain curve interpolation.



admissible path — tangent ≠ heading: not admissible

Figure 1: A unicycle-like robot starting along the horizontal direction can follow a smooth curve that bends toward $\boldsymbol{q}_f$ (left), but not the straight segment (right), whose tangent disagrees with the heading at $\boldsymbol{q}_i$.

Intuition

Think of parking a car: you cannot simply draw the straight line from where you are to the target spot, because the car cannot move sideways. The planner must hand the robot a curve it is *capable* of tracking. Obstacles are deliberately left out here; the whole difficulty already lives in the nonholonomic constraints.

2 Space-time separation *(slide 4)*

Definition 2.1 (Path, timing law, trajectory)**.** A configuration-space *trajectory* $\boldsymbol{q}(t)$, $t \in [t_i, t_f]$, with $\boldsymbol{q}(t_i) = \boldsymbol{q}_i$ and $\boldsymbol{q}(t_f) = \boldsymbol{q}_f$, can be split into:

- a geometric *path* $\boldsymbol{q}(s)$, where s is a path parameter running from $s_i = s(t_i)$ to $s_f = s(t_f)$;
- a *timing law* $s = s(t)$, strictly monotonic (increasing).

One possible choice of s is the arc length.

- **Path planning:** plan an admissible path from $\boldsymbol{q}_i$ to $\boldsymbol{q}_f$.
- **Trajectory planning:** plan an admissible trajectory from $\boldsymbol{q}_i$ to $\boldsymbol{q}_f$. It can be *direct*, or *indirect*: first plan a path, then add a timing law.

Intuition

A path is the *railway track*, the timing law is the *timetable*. The same track can be traveled quickly or slowly, but the track itself decides whether the train can follow it at all. This is why a robot that cannot track a given curve cannot be rescued by changing speed.

In symbols. A trajectory $\boldsymbol{q}(t)$, $t \in [t_i, t_f]$, with $\boldsymbol{q}(t_i) = \boldsymbol{q}_i$ and $\boldsymbol{q}(t_f) = \boldsymbol{q}_f$, is rewritten as the composition $\boldsymbol{q}(t) \;\to\; \boldsymbol{q}(s(t))$. The first factor, $\boldsymbol{q}(s)$ with $s \in [s_i, s_f]$, is the *path*: it carries only

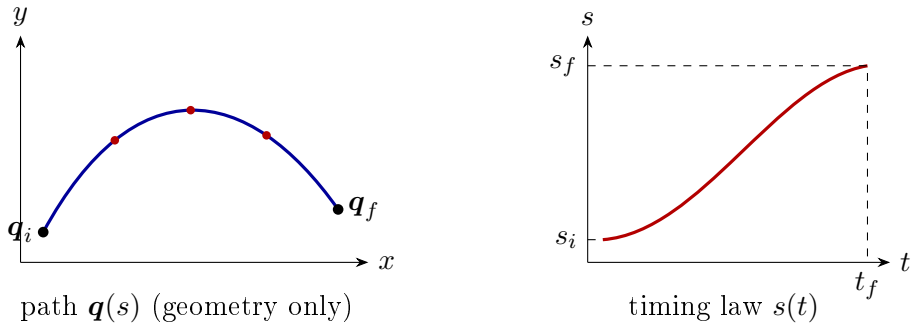


Figure 2: Space-time separation: the same path (left) can be run with many timing laws (right). The dots on the path are equally spaced in s, hence equally spaced in time only when $s(t)$ is linear.

the *geometry* of the motion. The second factor, $s(t)$ with $t \in [t_i, t_f]$, is the *timing law*: it says *when* each point of the path is reached.

3 Admissible paths and geometric model *(slides 5–8)*

3.1 Geometric path admissibility *(slides 5–6)*

Intuitively, the local mobility limitation caused by the nonholonomic constraints also shows up on *paths*: if a motion is not admissible, speeding up or slowing down will not make it admissible. To see this, start from the space-time separation:

$$\dot{\boldsymbol{q}} = \frac{\mathrm{d}\boldsymbol{q}}{\mathrm{d}t} = \frac{\mathrm{d}\boldsymbol{q}}{\mathrm{d}s}\,\dot{s} = \boldsymbol{q}'\,\dot{s}, \tag{1}$$

where the prime denotes differentiation with respect to s. The vector $\boldsymbol{q}'$ is directed as the *tangent* to the path in configuration space, and $\dot{s}$ only changes its modulus.

Example 3.1 (Tangent vector and arc length)**.** Let $\mathcal{C} = \mathbb{R}^2$, with coordinates $(q_1, q_2) = (x, y)$, and consider the path

$$x(s) = \sqrt{1 - s^2}, \qquad y(s) = s, \qquad s \in [0, 1],$$

a quarter of the unit circle going from $\boldsymbol{q}_i = (1, 0)$ to $\boldsymbol{q}_f = (0, 1)$. The tangent vector is

$$\boldsymbol{q}'(s) = \begin{pmatrix} x'(s) \\ y'(s) \end{pmatrix} = \begin{pmatrix} -s/\sqrt{1 - s^2} \\ 1 \end{pmatrix}, \qquad \|\boldsymbol{q}'(s)\| = \frac{1}{\sqrt{1 - s^2}} \neq 1,$$

so this parameter s is *not* the arc length (for which one would have $\|\boldsymbol{q}'\| = 1$, e.g., $x = \cos\sigma$, $y = \sin\sigma$).

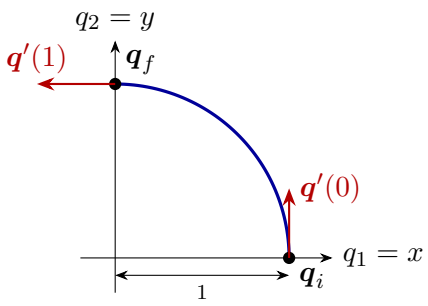


Figure 3: The path of Example 3.1: the tangent direction is the same for any parametrization, only the modulus of $\boldsymbol{q}'$ depends on s.

Intuition

Reparametrizing a path (e.g., from s to arc length σ) leaves the curve unchanged; it only rescales the tangent vector $\boldsymbol{q}'$. This is why, in the geometric model, only the *direction* of $\boldsymbol{q}'$ matters for admissibility, and why in all the planners below we are free to use a convenient s rather than the arc length.

Proposition 3.2 (Geometric path admissibility)**.** *A path $\boldsymbol{q}(s)$ is the support of an admissible trajectory (for any timing law with $\dot{s} > 0$) if and only if*

$$A(\boldsymbol{q})\,\boldsymbol{q}' = 0 \qquad \text{at all points of the path.} \tag{2}$$

Proof. Using (1), the constraints become $A(\boldsymbol{q})\dot{\boldsymbol{q}} = A(\boldsymbol{q})\boldsymbol{q}'\,\dot{s} = 0$. Since $\dot{s}(t) > 0$ for all t we may divide by $\dot{s}$, which gives $A(\boldsymbol{q})\boldsymbol{q}' = 0$; conversely, if $A(\boldsymbol{q})\boldsymbol{q}' = 0$ holds along the path, then $A(\boldsymbol{q})\dot{\boldsymbol{q}} = 0$ for any $\dot{s}$. □

Definition 3.3 (Geometric model and geometric inputs)**.** Let the columns of $G(\boldsymbol{q})$ span $\mathcal{N}(A(\boldsymbol{q}))$. All geometrically admissible paths are solutions of the *geometric model*

$$\boldsymbol{q}' = G(\boldsymbol{q})\,\tilde{\boldsymbol{u}}, \tag{3}$$

where $\tilde{\boldsymbol{u}}$ are the *geometric inputs*. They are related to the velocity inputs of the kinematic model by

$$\boldsymbol{u}(t) = \tilde{\boldsymbol{u}}(s)\,\dot{s}(t). \tag{4}$$

Once the geometric inputs $\tilde{\boldsymbol{u}}(s)$ are chosen, the path is determined; selecting a timing law $s = s(t)$ then identifies a particular trajectory along that path.

Kinematic versus geometric model. The geometric model is built exactly in parallel to the kinematic model (KM). Letting $\boldsymbol{g}_1, \dots, \boldsymbol{g}_m$ be the columns of $G(\boldsymbol{q})$:

$$\text{(KM)} \quad A(\boldsymbol{q})\dot{\boldsymbol{q}} = 0 \;\Rightarrow\; \dot{\boldsymbol{q}} \in \mathcal{N}(A(\boldsymbol{q})) \;\Rightarrow\; \dot{\boldsymbol{q}} = \sum_{i=1}^{m} \boldsymbol{g}_i(\boldsymbol{q})\,u_i, \tag{5}$$

$$\text{(GM)} \quad A(\boldsymbol{q})\boldsymbol{q}' = 0 \;\Rightarrow\; \boldsymbol{q}' \in \mathcal{N}(A(\boldsymbol{q})) \;\Rightarrow\; \boldsymbol{q}' = \sum_{i=1}^{m} \boldsymbol{g}_i(\boldsymbol{q})\,\tilde{u}_i. \tag{6}$$

Multiplying the geometric model by $\dot{s}$ and using $\dot{\boldsymbol{q}} = \boldsymbol{q}'\dot{s}$,

$$\dot{\boldsymbol{q}} = \boldsymbol{q}'\,\dot{s} = \sum_{i=1}^{m} \boldsymbol{g}_i(\boldsymbol{q})\,\tilde{u}_i\,\dot{s},$$

and comparing with the kinematic model (the $\boldsymbol{g}_i$ are linearly independent) gives $u_i = \tilde{u}_i\,\dot{s}$. The u_i are the *pseudovelocities* (or *velocity inputs*); the $\tilde{u}_i$ are the *geometric inputs*. In short: the kinematic model is a *trajectory generator*, the geometric model is a *path generator*.

Intuition

The condition $A(\boldsymbol{q})\boldsymbol{q}' = 0$ is the kinematic model with time "factored out". The nonholonomic constraints say which *directions* of motion are allowed; speed has no say in direction. So the geometric model is just the kinematic model run with a different clock: s instead of t, and $\tilde{\boldsymbol{u}}$ instead of $\boldsymbol{u}$.

3.2 The unicycle example *(slides 7–8)*

For the unicycle the pure-rolling constraint is $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$, i.e., $(\sin\theta \quad -\cos\theta \quad 0)\,\dot{\boldsymbol{q}} = 0$. Replacing $\dot{\boldsymbol{q}}$ by $\boldsymbol{q}'\dot{s}$ and dividing by $\dot{s} > 0$ gives $(\sin\theta \quad -\cos\theta \quad 0)\,\boldsymbol{q}' = 0$, the *path admissibility* condition, which reads

$$x'\sin\theta - y'\cos\theta = 0, \tag{7}$$

i.e., the Cartesian path tangent must align with the sagittal axis of the robot. The robot may travel along the tangent in either sense: forward motion if the heading points along the tangent, backward motion if it points opposite.

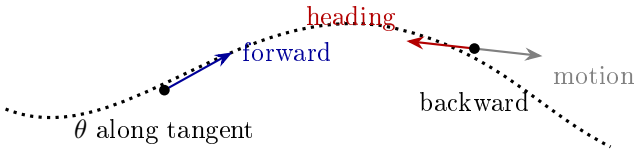


Figure 4: Along a Cartesian path the unicycle heading is tangent to the path; it points the same way as the motion (forward) or the opposite way (backward).

A Cartesian path with a *discontinuous tangent* is not admissible, unless the unicycle is allowed to stop at the discontinuity points to realign with the new tangent.

The geometric model of the unicycle is

$$\begin{aligned} x' &= \tilde{v}\cos\theta, \\ y' &= \tilde{v}\sin\theta, \\ \theta' &= \tilde{\omega}, \end{aligned} \tag{8}$$

with geometric inputs related to the velocity inputs by $v(t) = \tilde{v}(s)\,\dot{s}(t)$ and $\omega(t) = \tilde{\omega}(s)\,\dot{s}(t)$.

Example 3.4 (A Cartesian path with a corner)**.** Consider a Cartesian path made of a horizontal segment followed by a vertical one. Is it admissible for a unicycle? Two paths in the configuration space (x, y, θ) have this same *footprint* in the (x, y) plane:

(1) θ stays at 0 (or jumps at the corner). On the horizontal segment $y' = 0$ and $x'\sin 0 - y'\cos 0 = 0$ holds, but on the vertical segment $x' = 0$, $y' \neq 0$ and the condition fails; a jump of θ would make the path discontinuous in $\mathcal{C}$. **Not admissible.**

(2) $\theta = 0$ on the horizontal segment; at the corner θ goes from 0 to $\pi/2$ while $x' = y' = 0$ (rotation on the spot); then $\theta = \pi/2$ on the vertical segment. The condition holds at every point, so the path **is admissible**, but only if the unicycle *stops* at the corner.

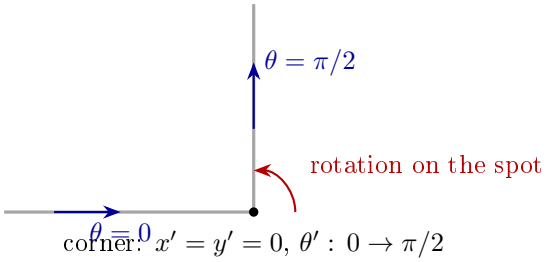


Figure 5: Path (2) of Example 3.4: the footprint has a corner, but the robot realigns with the new tangent by rotating at the corner.

Intuition

A corner in the Cartesian path is a place where the robot would have to change heading instantly. A unicycle can do that only by stopping and rotating on the spot, i.e., by "cheating" the pure path picture.

Planning formulation. Path planning for WMRs is generally formulated and solved with the *geometric* model. Trajectory planning can be solved *directly* with the kinematic model, or *indirectly* by first planning a geometric path and then choosing a timing law.

4 Flat outputs *(slides 9–13)*

4.1 Differential flatness *(slides 9–10)*

Definition 4.1 (Differential flatness)**.** The system $\dot{\boldsymbol{x}} = f(\boldsymbol{x}) + G(\boldsymbol{x})\boldsymbol{u}$ is *differentially flat* if there exists a set of outputs $\boldsymbol{y}$, called *flat outputs*, such that the state and the control inputs can be expressed *algebraically* as functions of $\boldsymbol{y}$ and its time derivatives:

$$\boldsymbol{x} = \boldsymbol{x}\big(\boldsymbol{y}, \dot{\boldsymbol{y}}, \ddot{\boldsymbol{y}}, \ldots, \boldsymbol{y}^{(r)}\big), \tag{9}$$

$$\boldsymbol{u} = \boldsymbol{u}\big(\boldsymbol{y}, \dot{\boldsymbol{y}}, \ddot{\boldsymbol{y}}, \ldots, \boldsymbol{y}^{(r)}\big). \tag{10}$$

The definition applies equally to *geometric* models if s replaces t and derivatives with respect to t are replaced by derivatives with respect to s. If the path (trajectory) of $\boldsymbol{y}$ is known, the path (trajectory) of $\boldsymbol{x}$ and the history of $\boldsymbol{u}$ in s (or t) can be *reconstructed*.

Intuition

Flat outputs are "free handles": pick any sufficiently smooth curve for $\boldsymbol{y}$, and everything else follows by differentiation, with no differential equation to integrate. For a unicycle, picking a smooth planar curve $(x(s), y(s))$ is enough: the heading is the direction of the curve, and the inputs are its speed and curvature.

Proposition 4.2 (Flatness of the unicycle and the bicycle)**.** *The unicycle and the bicycle are differentially flat with x and y as flat outputs. For the geometric model of the unicycle, a Cartesian path $(x(s), y(s))$ determines the path of the whole state $\boldsymbol{q}(s) = (x(s), y(s), \theta(s))$ and*

the geometric inputs:

$$\theta(s) = \arctan\frac{y'(s)}{x'(s)} = \operatorname{atan2}\big(y'(s), x'(s)\big) + k\pi, \tag{11}$$

$$\tilde{v}(s) = \pm\sqrt{(x'(s))^2 + (y'(s))^2}, \tag{12}$$

$$\tilde{\omega}(s) = \frac{y''(s)x'(s) - x''(s)y'(s)}{(x'(s))^2 + (y'(s))^2}. \tag{13}$$

For forward (backward) motion take $k = 1$ ($k = 0$) and the $+$ ($-$) sign. For the bicycle, let L be the wheelbase and delta the front-wheel steering angle; its kinematic model is

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \frac{v}{L}\tan\delta.$$

On intervals where the vehicle does not stop, the same Cartesian outputs reconstruct its state and inputs as

$$\begin{aligned}
\theta &= \operatorname{atan2}(\dot{y}, \dot{x}) \quad \textit{(modulo } \pi\textit{)},\\
v &= \pm\sqrt{\dot{x}^2 + \dot{y}^2},\\
\delta &= \arctan\left(L\,\frac{\operatorname{sgn}(v)(\dot{x}\ddot{y} - \dot{y}\ddot{x})}{(\dot{x}^2 + \dot{y}^2)^{3/2}}\right).
\end{aligned}$$

The sign of v selects forward or backward motion, with the corresponding heading branch.

Derivation for the unicycle. From $x' = \tilde{v}\cos\theta$ and $y' = \tilde{v}\sin\theta$ we get $(x')^2 + (y')^2 = \tilde{v}^2$, hence $\tilde{v} = \pm\sqrt{(x')^2 + (y')^2}$, and $y'/x' = \tan\theta$, hence $\theta = \arctan(y'/x')$ up to a multiple of π. Differentiating $\tan\theta = y'/x'$:

$$\frac{\theta'}{\cos^2\theta} = \frac{y''x' - x''y'}{(x')^2} \implies \tilde{\omega} = \theta' = \frac{y''x' - x''y'}{(x')^2}\cos^2\theta = \frac{y''x' - x''y'}{(x')^2 + (y')^2},$$

since $\cos^2\theta = (x')^2/\big((x')^2 + (y')^2\big)$. □

Derivation for the bicycle. The first two bicycle equations have the same form as the unicycle equations, so they give v and θ directly from (x, y) and their first derivatives, as above. Differentiating the heading yields

$$\dot{\theta} = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}.$$

Using $\dot{\theta} = (v/L)\tan\delta$ and $|v| = \sqrt{\dot{x}^2 + \dot{y}^2}$ (with the heading branch chosen consistently with the sign of v), we obtain

$$\tan\delta = L\frac{\operatorname{sgn}(v)(\dot{x}\ddot{y} - \dot{y}\ddot{x})}{(\dot{x}^2 + \dot{y}^2)^{3/2}},$$

which gives the steering reconstruction above. Thus state and inputs are algebraic functions of the flat outputs and their derivatives, proving flatness wherever $v \neq 0$. □

Intuition

$\tilde{v}$ is the "speed along the curve" and $\tilde{\omega}$ is $\tilde{v}$ times the *curvature* of the Cartesian path: the formula for $\tilde{\omega}$ is the familiar planar curvature formula $\kappa = (y''x' - x''y')/\big((x')^2 + (y')^2\big)^{3/2}$ multiplied by $\tilde{v}$. The sign choice only decides whether the robot runs the curve nose-first or tail-first.

4.2 Flatness and the chained form *(slides 11–12)*

Theorem 4.3 (Flatness, chained form, linearization)**.** *For driftless dynamic systems, differential flatness is a necessary and sufficient condition for transformability into chained form; hence all WMRs with dimension up to 4 are flat. For general dynamic systems, differential flatness is a necessary and sufficient condition for linearizability via dynamic feedback.*

Singular points of the reconstruction.

- If $x'(s) = y'(s) = 0$ at an isolated point (as at cusps or motion inversions), then $\tilde{v}(s) = 0$ there; at these points $\theta(s)$ and $\tilde{\omega}(s)$ can be computed by continuity arguments.
- The reconstruction of θ and $\tilde{\omega}$ *fails* if the Cartesian path degenerates to a point ($x'(s) = y'(s) = 0$ identically). In fact, the unicycle may then have any orientation at the contact point, and may even be rotating on the spot.

Geometrically: at an isolated cusp the Cartesian tangent vanishes only at the tip, so $\tilde{v} = 0$ there, the robot stops, inverts its motion and leaves along the other branch; if instead $x' \equiv y' \equiv 0$ over an interval, the contact point does not move and $\theta, \tilde{\omega}$ cannot be recovered from (x, y) (they are free in the geometric model $\theta' = \tilde{\omega}$).

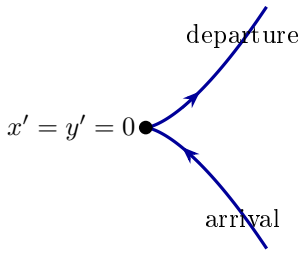


Figure 6: A cusp of the Cartesian path: the tangent vanishes only at the tip, where the unicycle inverts its motion.

Proposition 4.4 (Flatness of the $(2, n)$ chained form)**.** *The $(2, n)$ chained form is differentially flat, with flat outputs z_1 and z_n; all the other state variables and the associated control inputs can be computed from them.*

Example 4.5 (The $(2, 3)$ chained form)**.** Consider

$$\begin{aligned} z_1' &= \tilde{w}_1, \\ z_2' &= \tilde{w}_2, \\ z_3' &= z_2\, \tilde{w}_1. \end{aligned} \tag{14}$$

The reconstruction formulas are

$$z_2 = \frac{z_3'}{z_1'} \quad \text{(state)}, \qquad \tilde{w}_1 = z_1', \qquad \tilde{w}_2 = \frac{z_1' z_3'' - z_1'' z_3'}{(z_1')^2} \quad \text{(input)}. \tag{15}$$

Derivation. The first equation gives $\tilde{w}_1 = z_1'$. Substituting in the third, $z_3' = z_2 z_1'$, so $z_2 = z_3'/z_1'$ (when $z_1' \neq 0$). The second equation then gives $\tilde{w}_2 = z_2' = \big(z_3'' z_1' - z_3' z_1''\big)/(z_1')^2$ by the quotient rule. □

Intuition

In chained form the first variable z_1 acts as "clock": it advances at the rate $\tilde{w}_1$. Dividing the third equation by the first, $\mathrm{d}z_3/\mathrm{d}z_1 = z_2$: the slope of z_3 against z_1 *is* z_2. Choose the curve $z_3(z_1)$ and z_2 is forced; its derivative then gives the remaining input. This is exactly the same

"choose the curve, read off the rest" mechanism as for (x, y) in the unicycle.

4.3 An algorithm for planning with flat outputs *(slide 13)*

Reconstruction formulas associate admissible paths (trajectories) to the evolution of the flat outputs, which can be chosen *arbitrarily*. This leads to the following algorithm.

Algorithm 4.6 (Path planning via flat outputs).
1. Compute $\boldsymbol{y}_i$ and $\boldsymbol{y}_f$ corresponding to $\boldsymbol{q}_i$ and $\boldsymbol{q}_f$.
2. Find $\boldsymbol{y}(s)$ by interpolating $\boldsymbol{y}_i$ and $\boldsymbol{y}_f$ with the appropriate boundary conditions for $\boldsymbol{q}$.
3. Use the reconstruction formulas to find $\boldsymbol{q}(s)$ and $\tilde{\boldsymbol{u}}(s)$.

The resulting path is *automatically* admissible, i.e., it satisfies the nonholonomic constraints at all points. The same algorithm can be used for *trajectory planning* by interpolating over t rather than over s.

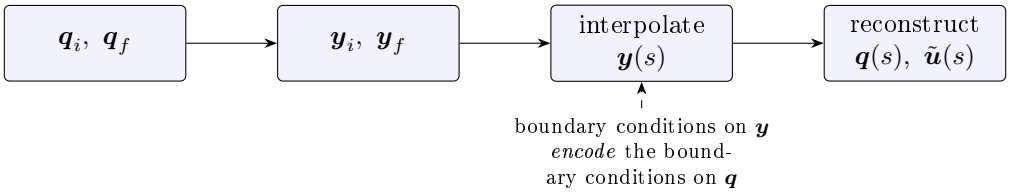


Figure 7: Planning with flat outputs: admissibility is built into the reconstruction step.

Intuition

Normally planning for a nonholonomic robot means solving a constrained problem. Flatness turns it into an *unconstrained* one: the constraints are already "absorbed" by the reconstruction formulas. The only subtlety is that orientation (and higher-order variables) are *not* flat outputs, so their boundary values must be translated into *derivative* conditions on $\boldsymbol{y}$; this is the role of the "appropriate boundary conditions" in step 2.

5 Unicycle path planning via Cartesian polynomials *(slides 14–15)*

We apply Algorithm 4.6 to the unicycle, whose flat outputs are x and y. Given $\boldsymbol{q}_i = (x_i, y_i, \theta_i)$ and $\boldsymbol{q}_f = (x_f, y_f, \theta_f)$, find an admissible path $\boldsymbol{q}(s)$, $s \in [s_i, s_f]$, with $\boldsymbol{q}(s_i) = \boldsymbol{q}_i$ and $\boldsymbol{q}(s_f) = \boldsymbol{q}_f$.

Let $s_i = 0$, $s_f = 1$ and use *pre-fitted cubic polynomials*

$$x(s) = s^3 x_f - (s-1)^3 x_i + \alpha_x s^2 (s-1) + \beta_x s (s-1)^2, \tag{16}$$
$$y(s) = s^3 y_f - (s-1)^3 y_i + \alpha_y s^2 (s-1) + \beta_y s (s-1)^2, \tag{17}$$

which already satisfy the boundary conditions on x, y (indeed $x(0) = x_i$, $x(1) = x_f$). Since θ_i and θ_f are given, we must also impose

$$\arctan \frac{y'(0)}{x'(0)} = \theta_i, \qquad \arctan \frac{y'(1)}{x'(1)} = \theta_f. \tag{18}$$

Since the interpolation is done independently in x and y, these are separated as

$$\begin{aligned} x'(0) &= k_i \cos\theta_i, \quad x'(1) = k_f \cos\theta_f, \\ y'(0) &= k_i \sin\theta_i, \quad y'(1) = k_f \sin\theta_f, \end{aligned} \tag{19}$$

where $k_i \neq 0$, $k_f \neq 0$ are free parameters. In other words, the flat outputs of the boundary configurations are (x_i, y_i) and (x_f, y_f), while the orientations θ_i, θ_f translate into conditions on the first derivatives. Each of $x(s)$ and $y(s)$ thus carries four boundary conditions (value and derivative at both ends), which matches the four coefficients of a cubic.

Proposition 5.1 (Cartesian cubic planner). *Under the simplifying assumption $k_i = k_f = k$, the additional boundary conditions give*

$$\begin{aligned} \alpha_x &= k\cos\theta_f - 3x_f, \quad \beta_x = k\cos\theta_i + 3x_i, \\ \alpha_y &= k\sin\theta_f - 3y_f, \quad \beta_y = k\sin\theta_i + 3y_i. \end{aligned} \tag{20}$$

Moreover $\tilde{v}(0) = \tilde{v}(1) = k$; $k > 0$ gives forward motion, $k < 0$ backward motion.

Proof. Differentiating, $x'(s) = 3s^2 x_f - 3(s-1)^2 x_i + \alpha_x(3s^2 - 2s) + \beta_x\big((s-1)^2 + 2s(s-1)\big)$. Then $x'(0) = -3x_i + \beta_x = k\cos\theta_i$ and $x'(1) = 3x_f + \alpha_x = k\cos\theta_f$, which give β_x, α_x. The computation for y is identical. At the extremes $\tilde{v} = \pm\sqrt{(x')^2 + (y')^2} = \pm|k|$, with the sign of k for the chosen direction of motion. □

From $x(s)$ and $y(s)$ one obtains $\theta(s)$, $\tilde{v}(s)$ and $\tilde{\omega}(s)$ via the reconstruction formulas of Proposition 4.2.

Intuition

This is cubic *Hermite interpolation*: (x_i, y_i) and (x_f, y_f) are the endpoints, and the vectors $k(\cos\theta_i, \sin\theta_i)$ and $k(\cos\theta_f, \sin\theta_f)$ are the end tangents. The heading is forced to be tangent to the path (so the path is admissible), and k is a "tangent length": a large k makes the robot leave $\boldsymbol{q}_i$ and arrive at $\boldsymbol{q}_f$ *straight and for a long stretch* before bending in the middle, a small k gives a more direct, tighter curve.

Example 5.2 (Worked numbers: forward parking). Take $\boldsymbol{q}_i = (0, 0, 0)$ and $\boldsymbol{q}_f = (4, -3, -\pi/2)$. Then $\alpha_x = -12$, $\beta_x = k$, $\alpha_y = 9 - k$, $\beta_y = 0$, so

$$x(s) = (k-8)s^3 + (12-2k)s^2 + ks, \qquad y(s) = (6-k)s^3 - (9-k)s^2.$$

One checks $x'(0) = k$, $y'(0) = 0$ (heading 0, speed k) and $x'(1) = 0$, $y'(1) = -k$ (heading $-\pi/2$, speed k). At the start, $\tilde{\omega}(0) = y''(0)/k = 2(k-9)/k$: for $k = 5$ this is -1.6 (the robot immediately steers right), whereas for $k = 20$ it is $+1.1$ (the robot first swings slightly *left*, up, and then comes down).

Intuition

The sign change of $\tilde{\omega}(0)$ at $k = 9$ is the numerical face of "long tangents overshoot": with a very large k the cubic is stretched along the initial heading, so it first bulges forward and upward before turning down to the target.

6 Unicycle path planning via the chained form *(slides 16–17)*

As an alternative one may transform the unicycle into the $(2, 3)$ chained form, whose flat outputs are z_1 and z_3. (The standard unicycle transformation is $z_1 = \theta$, $z_2 = x\cos\theta + y\sin\theta$, $z_3 = x\sin\theta - y\cos\theta$, with inputs $w_1 = \omega$, $w_2 = v - z_3\omega$.)

First compute the initial and final values of z_1, z_2, z_3 corresponding to $\boldsymbol{q}_i$ and $\boldsymbol{q}_f$. Then,

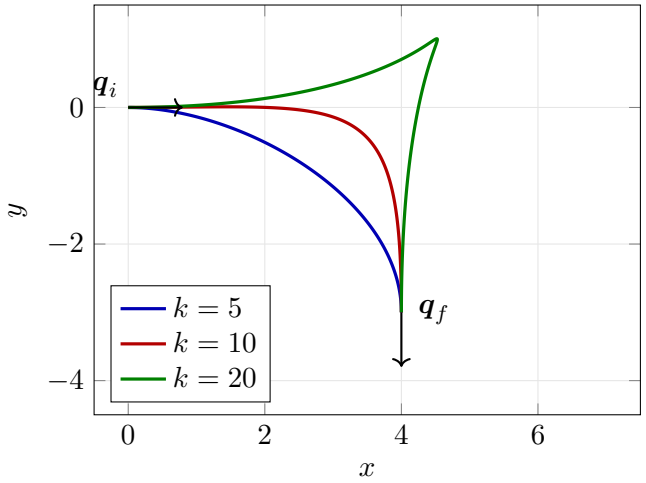


Figure 8: Cartesian cubic planner for the forward-parking problem of Example 5.2, for increasing k (own computation from the formulas above). Larger k produces longer take-off and landing phases.

assuming $z_{1,i} \neq z_{1,f}$, define z_1 and z_3 over $[0, 1]$ as

$$z_1(s) = z_{1,f}\, s - (s-1)\, z_{1,i}, \tag{21}$$

$$z_3(s) = s^3 z_{3,f} - (s-1)^3 z_{3,i} + \alpha_3 s^2(s-1) + \beta_3 s(s-1)^2, \tag{22}$$

which already satisfy the boundary conditions on z_1, z_3. Since $z_{2,i}$ and $z_{2,f}$ are also given, we must impose

$$\frac{z_3'(0)}{z_1'(0)} = z_{2,i}, \qquad \frac{z_3'(1)}{z_1'(1)} = z_{2,f}. \tag{23}$$

Proposition 6.1 (Chained-form cubic planner). *Imposing these conditions gives*

$$\alpha_3 = z_{2,f}(z_{1,f} - z_{1,i}) - 3z_{3,f}, \qquad \beta_3 = z_{2,i}(z_{1,f} - z_{1,i}) + 3z_{3,i}. \tag{24}$$

Proof. $z_1'(s) = z_{1,f} - z_{1,i} =: \Delta$ is constant. As in Proposition 5.1, $z_3'(0) = -3z_{3,i} + \beta_3$ and $z_3'(1) = 3z_{3,f} + \alpha_3$; setting them equal to $z_{2,i}\Delta$ and $z_{2,f}\Delta$ respectively gives the result. □

From $z_1(s)$ and $z_3(s)$ one obtains $z_2(s)$ and $\tilde{w}_1(s)$, $\tilde{w}_2(s)$ via the reconstruction formulas of Example 4.5. Finally $\boldsymbol{q}(s)$, $\tilde{v}(s)$ and $\tilde{\omega}(s)$ are computed by inverting the chained form coordinate and input transformations.

The singular case. If $z_{1,i} = z_{1,f}$ (i.e., $\theta_i = \theta_f$) the construction breaks down (z_1 would be constant). One can introduce a *via point* $\boldsymbol{q}_v = (x_v, y_v, \theta_v)$ with $\theta_v \neq \theta_i$ and split the original problem in two; alternatively, set $z_{1,f} = z_{1,i} + 2\pi$ (a full turn).

Direct trajectory planning. Both previous schemes (Cartesian polynomials and chained form) can be used for direct trajectory planning: just let $s = t$ and use the kinematic model.

Intuition

Here $z_1 = \theta$ is used as the interpolation "clock" and it moves *linearly* between θ_i and θ_f: the heading changes monotonically. That is why $\theta_i = \theta_f$ is a problem: the clock does not move at all and the curve $z_3(z_1)$ has nothing to be a function of. Adding a via point or a full 2π turn restores a clock that ticks.

Remark on the numerical results. For the unicycle chained form, the position enters z_2, z_3 through the current (x, y). Placing the origin of z_2, z_3 at $\boldsymbol{q}_i$ (so that $z_{2,i} = z_{3,i} = 0$) changes the plan: with the global origin a reorientation generates a Cartesian displacement, whereas with the origin at $\boldsymbol{q}_i$ pure rotations on the spot are representable (in the numerical check for $\boldsymbol{q}_i = (1, 1, 0) \to \boldsymbol{q}_f = (1, 1, \pi/2)$ with origin at $\boldsymbol{q}_i$ the integration gives $c_0 = c_1 = 0$, i.e. $x \equiv y \equiv$ const).

7 Path planning via parametrized inputs *(slides 18–21)*

7.1 Chained form with parametrized inputs *(slides 18–20)*

A different approach, not directly based on flatness, consists in writing the inputs in *parametrized form* and then imposing that the robot goes from $\boldsymbol{q}_i$ to $\boldsymbol{q}_f$. Assume the robot can be put in chained form,

$$\begin{aligned} z_1' &= \tilde{w}_1, \\ z_2' &= \tilde{w}_2, \\ z_3' &= z_2 \tilde{w}_1, \\ &\vdots \\ z_n' &= z_{n-1} \tilde{w}_1, \end{aligned} \tag{25}$$

which becomes *linear* (hence easily integrable) if $\tilde{w}_1(s)$ is constant.

Let $\Delta = z_{1,f} - z_{1,i}$ and choose the geometric inputs for $s \in [s_i, s_f] = [0, |\Delta|]$ as

$$\tilde{w}_1 = \operatorname{sign}(\Delta), \tag{26}$$

$$\tilde{w}_2 = c_0 + c_1 s + \cdots + c_{n-2} s^{n-2}. \tag{27}$$

Integrating the chained-form equations from $\boldsymbol{z}(0) = \boldsymbol{z}_i$ and imposing $\boldsymbol{z}(|\Delta|) = \boldsymbol{z}_f$ leads to a *linear system*

$$D(\Delta) \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-2} \end{pmatrix} = \boldsymbol{d}(\boldsymbol{z}_i, \boldsymbol{z}_f, \Delta), \tag{28}$$

with $D(\Delta)$ invertible if $\Delta \neq 0$.

Proposition 7.1 (The $(2, 3)$ case)**.** *For the $(2, 3)$ chained form,*

$$D = \begin{pmatrix} |\Delta| & \dfrac{\Delta^2}{2} \\ \operatorname{sign}(\Delta)\dfrac{\Delta^2}{2} & \dfrac{\Delta^3}{6} \end{pmatrix}, \qquad \boldsymbol{d} = \begin{pmatrix} z_{2,f} - z_{2,i} \\ z_{3,f} - z_{3,i} - z_{2,i}\Delta \end{pmatrix}, \tag{29}$$

and $\det D = -\operatorname{sign}(\Delta)\, \Delta^4/12 \neq 0$ *whenever* $\Delta \neq 0$.

Proof. With $\tilde{w}_1 = \operatorname{sign}(\Delta)$, $\tilde{w}_2 = c_0 + c_1 s$: $z_1(s) = z_{1,i} + \operatorname{sign}(\Delta) s$, which reaches $z_{1,f}$ at $s = |\Delta|$; $z_2(s) = z_{2,i} + c_0 s + c_1 s^2/2$; and $z_3' = \operatorname{sign}(\Delta) z_2$ gives $z_3(s) = z_{3,i} + \operatorname{sign}(\Delta)\big(z_{2,i} s + c_0 s^2/2 + c_1 s^3/6\big)$. Imposing $z_2(|\Delta|) = z_{2,f}$ gives $c_0 |\Delta| + c_1 \Delta^2/2 = z_{2,f} - z_{2,i}$. Imposing $z_3(|\Delta|) = z_{3,f}$ and using $\operatorname{sign}(\Delta)|\Delta| = \Delta$, $\operatorname{sign}(\Delta)|\Delta|^3 = \Delta^3$ gives $\operatorname{sign}(\Delta)\frac{\Delta^2}{2} c_0 + \frac{\Delta^3}{6} c_1 = z_{3,f} - z_{3,i} - z_{2,i}\Delta$. Finally $\det D = |\Delta|\Delta^3/6 - \operatorname{sign}(\Delta)\Delta^4/4 = -\operatorname{sign}(\Delta)\Delta^4/12$. □

If $\Delta = 0$ ($z_{1,i} = z_{1,f}$) a singular case is met, which can be handled as before (e.g., with a via point). Once planning on the chained form is complete, $\boldsymbol{q}(s)$ and $\tilde{\boldsymbol{u}}(s)$ for the original model

are computed via the inverse coordinate and input transformations. Although this method does not make explicit use of flat outputs, it still relies on transformability in chained form, which is strongly related to flatness (Theorem 4.3).

Intuition

With $\tilde{w}_1$ constant, s simply counts how far the "clock" variable z_1 has advanced, and the remaining equations are a *chain of integrators* driven by the polynomial $\tilde{w}_2$. The $n-1$ unknown coefficients are exactly as many as the $n-1$ boundary conditions on $z_2, \dots, z_n$ (those on z_1 are met by the choice of the interval $[0, |\Delta|]$), so a square linear system results. For $n = 3$ there are two coefficients (c_0, c_1) for two conditions.

7.2 Other parametrizations and trajectory planning *(slide 21)*

Other classes of parametrized inputs can be used, such as sinusoidal and piecewise-constant functions. Note that in *all* the presented path planning methods the path parameter s *does not represent the arc length*. This scheme too can be used for direct trajectory planning by replacing s with t and using the kinematic model in place of the geometric model.

8 Parking a unicycle: numerical results *(slides 22–27)*

8.1 Planner based on Cartesian polynomials *(slides 22–24)*

- **Forward parking** ($k = 5 > 0$): the motion is forward; a negative k would produce a backward motion. There are no motion inversions (a general result for this planner).
- **Effect of k:** increasing k (e.g., $k = 10$ and $k = 20$) leads to elongated take-off and landing phases; still no motion inversions.
- **Parallel parking** and **reorientation** (again $k = 10$ and $k = 20$): since the planner is based on flat outputs, a Cartesian path of *finite length* is needed for *any* kind of reconfiguration.

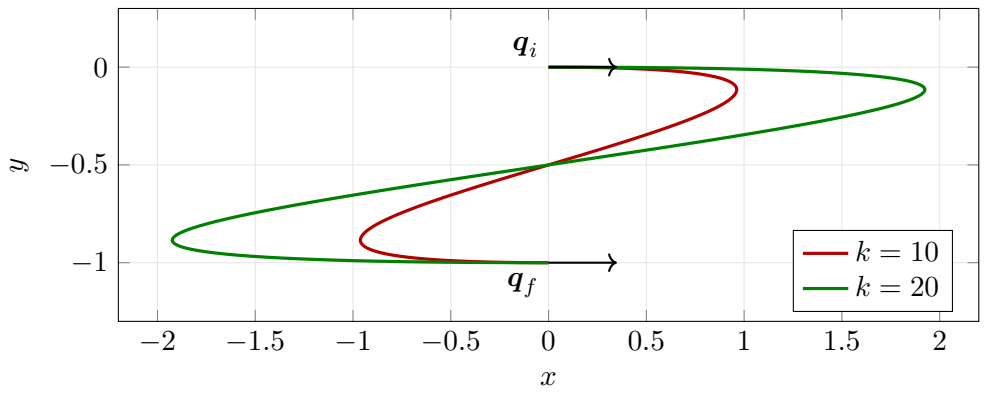


Figure 9: Parallel parking with the Cartesian planner, $\boldsymbol{q}_i = (0, 0, 0) \to \boldsymbol{q}_f = (0, -1, 0)$ (own computation). A larger k widens the forward and backward excursions needed to shift sideways.

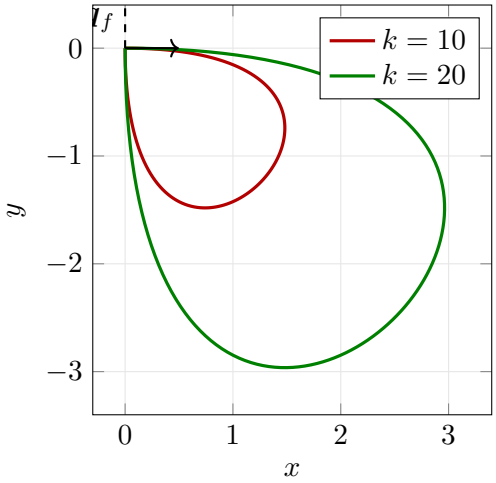


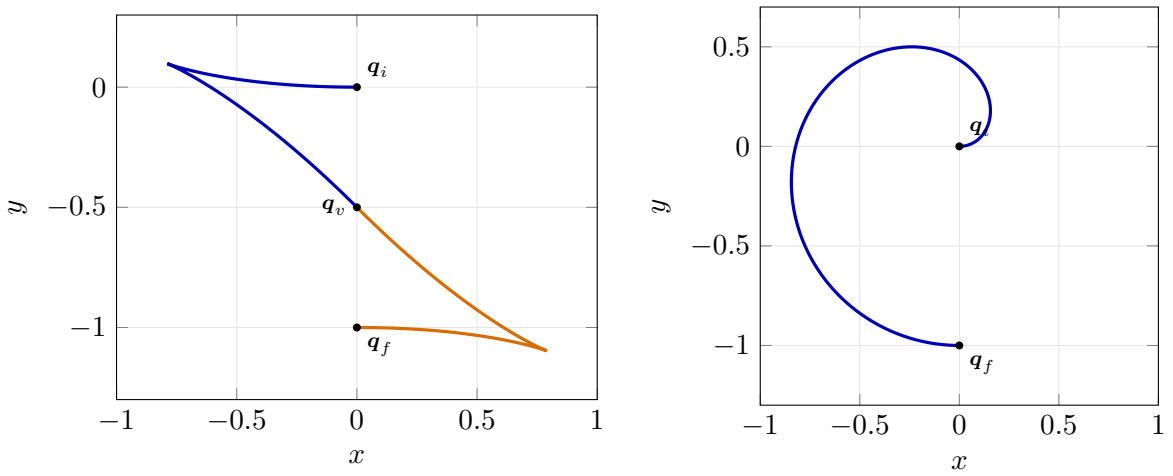
Figure 10: Reorientation $\boldsymbol{q}_i = (0, 0, 0) \rightarrow \boldsymbol{q}_f = (0, 0, \pi/2)$ with the Cartesian planner (own computation): the robot leaves heading east (solid arrow), draws a finite loop and returns heading north (dashed arrow).

Intuition

Reorienting a unicycle in place is easy for the physical robot (rotate on the spot), but the Cartesian planner has no way to say "stay at a point". The reconstruction formulas fail when the Cartesian path degenerates to a point (Section 4), so the planner must draw a small loop in the plane to change orientation.

8.2 Planner based on parametrized inputs *(slides 25–27)*

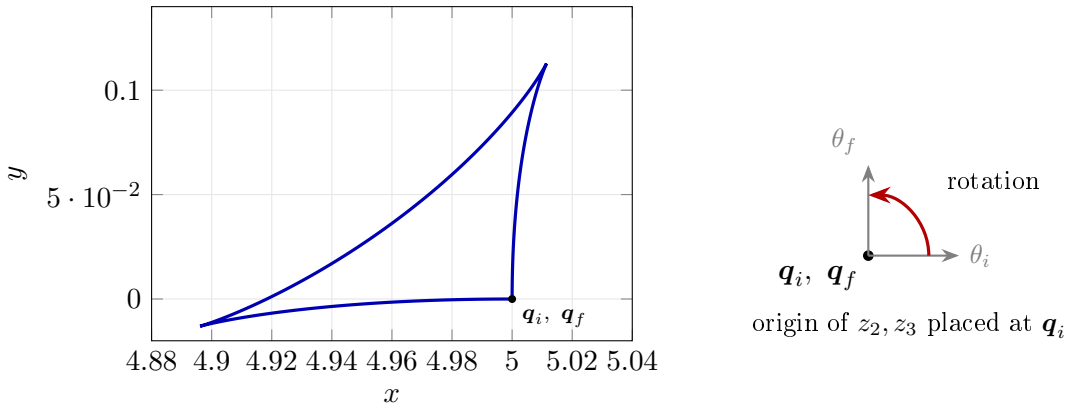
- **Forward parking:** the first maneuver is similar to that of the other planner. In the second maneuver a *cusp* appears: this planner can only generate paths along which the orientation varies monotonically.
- **Parallel parking:** the problem is solved with a via point, or by imposing a full rotation.
- **Reorientation:** using the classical chained form transformation, a Cartesian displacement is generated; placing the origin of z_2, z_3 at $\boldsymbol{q}_i$ avoids it.



with a via point

imposing a full rotation ($z_{1,f} = z_{1,i} + 2\pi$)

Figure 11: Parallel parking $\boldsymbol{q}_i = (0, 0, 0) \to \boldsymbol{q}_f = (0, -1, 0)$ with the parametrized-input planner. Since $\theta_i = \theta_f$ ($\Delta = 0$), the problem is either split at a via point (here $\boldsymbol{q}_v = (0, -0.5, -\pi/4)$) or solved with a full rotation of the orientation (own computation).



classical transformation (origin of z_2, z_3 elsewhere)

origin of z_2, z_3 placed at $\boldsymbol{q}_i$

Figure 12: Reorientation $\boldsymbol{q}_i = (x, y, 0) \to \boldsymbol{q}_f = (x, y, \pi/2)$ with the parametrized-input planner (own computation). With the classical transformation a small Cartesian displacement is generated (note the scale of the axes); placing the origin at $\boldsymbol{q}_i$ gives $c_0 = c_1 = 0$, i.e., a pure rotation on the spot.

9 Choice of timing law *(outline item)*

This item appears in the outline of the lecture; the slides provided end with the parking examples, so only the basic idea is given here (supplementary material, not taken from the slides).

Given a geometric path $\boldsymbol{q}(s)$ with inputs $\tilde{\boldsymbol{u}}(s)$, any $s = s(t)$ with $\dot{s} > 0$ yields an admissible trajectory with inputs $v(t) = \tilde{v}(s)\dot{s}$, $\omega(t) = \tilde{\omega}(s)\dot{s}$. Velocity bounds $|v| \le v_{\max}$, $|\omega| \le \omega_{\max}$ become

$$\dot{s}(t) \le \min\left\{ \frac{v_{\max}}{|\tilde{v}(s)|},\ \frac{\omega_{\max}}{|\tilde{\omega}(s)|} \right\},$$

so the timing law can be chosen, for instance, as the fastest one compatible with these bounds, or as a smooth profile (e.g., cubic) satisfying boundary conditions on s and $\dot{s}$.

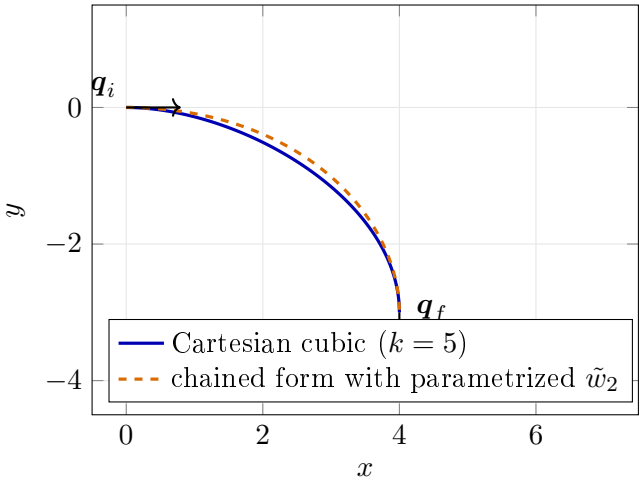


Figure 13: The two planners on the same forward-parking problem $\boldsymbol{q}_i = (0, 0, 0) \to \boldsymbol{q}_f = (4, -3, -\pi/2)$ (own computation). The parametrized-input planner turns monotonically from 0 to $-\pi/2$ with $\tilde{\omega} = \pm 1$ per unit of s, so s is the angle swept, not the arc length.

Intuition

Slowing down scales *both* inputs by the same factor, so the shape of the path never changes; it is the sharp parts of the path (large $|\tilde{\omega}|$) that force the robot to slow down.

10 Summary

	Cartesian polynomials	**Chained form (cubic z_3)**	**Parametrized inputs**
Flat outputs used	x, y	z_1, z_3	not directly (needs chained form)
What is chosen	cubic $x(s), y(s)$; k free	z_1 linear, z_3 cubic	$\tilde{w}_1 = \operatorname{sign}\Delta$, $\tilde{w}_2$ polynomial
Unknowns	$\alpha_x, \beta_x, \alpha_y, \beta_y$ (closed form), k	α_3, β_3 (closed form)	$c_0, \ldots, c_{n-2}$ (linear system $D\boldsymbol{c} = \boldsymbol{d}$)
Singular case	—	$\theta_i = \theta_f$	$\Delta = 0$
Singular case remedy	—	via point or $+2\pi$	via point or $+2\pi$
Orientation	follows the Cartesian curve	monotonic	monotonic
Cusps	none	may appear	may appear
s is arc length?	no	no	no
Direct trajectory planning	$s = t$	$s = t$	$s = t$

Table 1: Comparison of the three planners for the unicycle (all assume obstacle-free motion).

Key take-aways

- A path is admissible for a nonholonomic WMR iff $A(\boldsymbol{q})\boldsymbol{q}' = 0$; this is independent of the timing law, which only converts a path into a trajectory ($\boldsymbol{u} = \tilde{\boldsymbol{u}}\,\dot{s}$).

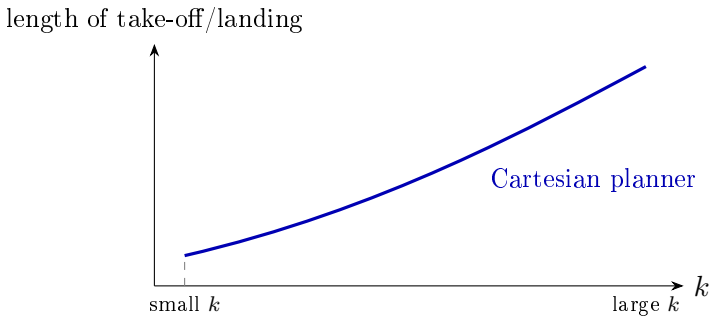


Figure 14: Qualitative trade-off in the Cartesian planner: larger k gives longer straight take-off and landing phases and a possible initial overshoot (qualitative sketch, not measured data).

- Differentially flat systems (unicycle, bicycle, all WMRs up to dimension 4) can be planned by interpolating the flat outputs and reconstructing the rest: admissibility comes for free.
- For the unicycle, a cubic Cartesian Hermite curve with tangent length k gives a closed-form planner; the sign of k selects forward or backward motion and a large k elongates take-off and landing.
- In chained form, either interpolate z_1, z_3 or parametrize $\tilde{w}_2$ and solve a linear system; both fail when $\theta_i = \theta_f$ and need a via point or a 2π turn, and both produce monotonically varying orientation.
- In none of the methods does s represent the arc length, and every scheme becomes a direct trajectory planner by setting $s = t$.